

Föreläsning 15

Ändrad 16 Oct 21:35

Indexerade variabler

Följande program lagrar de femton första *Fibonaccitalen*.
Och skriver sedan ut dem.

Ett *Fibonaccital* är summan av de två föregående
Fibonaccitalen. De två första Fibonaccitalen är bägge ett.

Här har ni dem i utskriften från programmet. Numrerade
från noll (i C numrerar man alltid från noll):

```

1  fibonacci[ 0] = 1
2  fibonacci[ 1] = 1
3  fibonacci[ 2] = 2
4  fibonacci[ 3] = 3
5  fibonacci[ 4] = 5
6  fibonacci[ 5] = 8
7  fibonacci[ 6] = 13
8  fibonacci[ 7] = 21
9  fibonacci[ 8] = 34
10 fibonacci[ 9] = 55
11 fibonacci[10] = 89
12 fibonacci[11] = 144
13 fibonacci[12] = 233
14 fibonacci[13] = 377
15 fibonacci[14] = 610

```

Programmet lagrar talen i en *indexerad variabel* --
`fibonacci`.

```

1  #include <stdio.h>
2
3  void main()
4  {
5      int fibonacci[15];
6      int i;
7
8      fibonacci[0] = 1;
9      fibonacci[1] = 1;
10     fibonacci[2] = fibonacci[0] + fibonacci[1];
11     fibonacci[3] = fibonacci[1] + fibonacci[2];
12     fibonacci[4] = fibonacci[2] + fibonacci[3];
13
14     for (i = 5; i < 15; i++)
15         fibonacci[i] = fibonacci[i-2] + fibonacci[i-1];
16
17     for (i = 0; i < 15; i++)
18         printf("fibonacci[%2d] = %3d \n", i, fibonacci[i]);
19 }

```

På rad 5 deklareraras femton stycken variabler. De heter
`fibonacci[0]`, `fibonacci[1]`, `fibonacci[2]`, osv till
`fibonacci[14]`.

Indexerade från noll till fjorton, alltså femton stycken.

(Kärt barn har många namn: Indexerad variabel, fält,
vektor, *array*, tabell...)

De enskilda variablerna i fältet kallar vi oftast
element, ibland *indexerade variabler*.

Hederliga heltalsvariabler

Dessa femton variabler är vanliga hederliga heltalsvariabler. Därför kan jag skriva ut dem med `printf()` på rad 18.

Jag kan också tilldela dem värden, och jag kan använda dem i aritmetiska uttryck. Bådadera görs i programmet ovan.

Och så vidare. Tja, till exempel, om jag velat, hade jag kunnat läsa in dem med `scanf`:

```
scanf("%d%d", &fibonacci[7], &fibonacci[9]);
```

Utan problem! Eller varför inte öka nån av dem med ett:

```
fibonacci[5]++;
```

Vanliga hederliga heltalsvariabler! Femton stycken.

Ligger tillsammans i minnet

En mycket viktig detalj är att de femton variablerna ligger efter varandra i minnet. Om `fibonacci[0]` ligger på adress

```
444001--444004,
```

(en `int` tar fyra byte) så ligger `fibonacci[1]` på adress

```
444005--444008.
```

Och så vidare. Det här betyder att om vi vet var en av dem ligger, kan vi alltid räkna ut var de andra ligger i minnet.

Den kunskapen utnyttjas friskt i programspråket C.

Om vi skulle vilja låta en funktion ändra värde på flera indexerade variabler, så skulle vi kunna nöja oss med att ge adressen till den första som parameter till funktionen -- `&fibonacci[0]`.

Sedan kan funktionen själv räkna ut var de andra ligger.

Låt oss försöka! Vi deklarerar en indexerad variabel, och låter en funktion sätta den första till summan av de övriga.

Men vi berättar inte för funktionen var de övriga ligger lagrade.

Vi skickar bara adressen till den första som argument.

```
1  #include <stdio.h>
2
3  void summera(int *ptr)
4  {
5      *ptr = *(ptr + 1) + *(ptr + 2);
6  }
7
8  void main()
9  {
10     int nr[3] = {0,100,200};
11
12     summera(&nr[0]);
13
14     printf("nr[0] = nr[1] + nr[2] = %d \n", nr[0]);
15 }
```

Det finns en finess i deklarationen av `nr` på rad 10. Vi initierar `nr`. Initieringar i deklarationen har ni ju sett förut.

Det nya är syntaxen: mjuka parenteser, kommatecken.

Programmet fungerar:

```
1 nr[0] = nr[1] + nr[2] = 300
```

Hur kan det göra det? undrar den observante läsaren. Tar inte en `int` upp fyra byte? Borde inte rad 5 lyda

```
*ptr = *(ptr + 4) + *(ptr + 8)
```

Ja, jo. I och för sig. Jo, det stämmer. Där ligger de. Om man använder vanligt + från vanlig aritmetik vill säga.

Men det gör man inte alltid i C.

Pekararitmetik

Operatorm + fungerar annorlunda tillsammans med pekare. Programmeraren skriver

```
ptr + 7
```

Därefter omvandlar kompilatorn det hela till

```
ptr + 7 * sizeof(*ptr)
```

Den speciella pekararitmetiken gäller vid addition och subtraktion av adresser.

Vanligt index behövs egentligen ej

I själva verket behövs inte det vanliga sättet att skriva:

```
nr[2]
```

eftersom vi alltid kan skriva

```
*(nr + 2)
```

istället.

Så ligger det till. Den vanliga indexformen finns bara av praktiska skäl. För att den ger mindre att skriva. Man kan klara sig utan den.

Och nr och fibonacci är bara adresser

Denna rad:

```
int nr[3];
```

betyder egentligen följande:

■ Plats reserveras för tre heltal. I följd, intill varandra i minnet, efter varandra.

■ `nr` får användas som namn på adressen till det första heltalet: `nr = &nr[0]`.

När vi sedan i programmet skriver

```
nr[2]
```

översätter kompilatorn detta till

```
*(nr + 2) = *(&nr[0] + 2)
```

Arrangemanget fungerar eftersom de femton variablerna garanterat ligger efter varandra i minnet. [Här är en illustration](#) av var variablerna lagras.

Olika synsätt på indexerade variabler

Å andra sidan. Hur man ska tänka när man väl använder indexerade variabler -- det är en annan historia.

Då behöver man normalt aldrig bry sig om "pekare", "adresser", "pekararitmetik". Eller bry sig om att elementen lagras efter varandra.

Nej, då backar man tillbaka till det första jag sa. Att det som händer vid deklarationen är att vi skapar vanliga variabler med namn som `nr[0]`, `nr[1]`, etc.

Funktionen ovan -- `summera()` -- den skriver man hellre såhär:

```
void summera(int ptr[])
{
    ptr[0] = ptr[1] + ptr[2];
}
```

Indexhakarna efter `ptr` är ett snyggt sätt att berätta att pekaren som är argument kommer att användas som en indexerad variabel.

Man skulle nog för övrigt inte kalla parametern `ptr`, utan snarare `nr`.

Anropet: Eftersom `nr == &nr[0]` brukar anropet se ut såhär:

```
summera(nr).
```

Den som har erfarenhet från andra programspråk kan låtsas att `nr` är en *variabelparameter* (se [förra föreläsningen](#)).

Nästa lektion

Programmeringsmetoder för maskinvara 1997

